Lab 3

Generated by Doxygen 1.9.1

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 3

Class Documentation

3.1 LCG Class Reference

Linear Congruential Generator (LCG) random number generator.

`#include <func.h>`

Public Member Functions

- LCG (uint32_t seed)

  Generates the next random number in sequence.
- uint32_t next ()

  Generates the next random number in sequence.

3.1.1 Constructor & Destructor Documentation

3.1.1.1 LCG()

```
LCG::LCG (
            uint32_t seed )
```

Constructs a Linear Congruential Generator with given seed.

Returns

Next 32-bit unsigned random number

Parameters

seed	Initial seed value

3.1.2 Member Function Documentation

3.1.2.1 next()

```
uint32_t LCG::next ( )
```

Generates next random number using LCG algorithm.

Returns

Next 32-bit unsigned random number

32-bit unsigned random number

The documentation for this class was generated from the following files:

- func.h
- func.cpp

3.2 MiddleSquare Class Reference

Middle Square random number generator.

```
#include <func.h>
```

Public Member Functions

- MiddleSquare (uint32_t seed)
  Constructor that initializes the generator with a seed.
- uint32_t next ()
  Generates the next random number using middle-square method.

3.2.1 Constructor & Destructor Documentation

3.2.1.1 MiddleSquare()

```
MiddleSquare::MiddleSquare (
            uint32_t seed )
```

Constructs a Middle Square generator with given seed.

Parameters

seed	Initial seed value

3.2.2 Member Function Documentation

3.2.2.1 next()

```
uint32_t MiddleSquare::next ( )
```

Generates next random number using Middle Square method.

Returns

Next 32-bit unsigned random number

32-bit unsigned random number

The documentation for this class was generated from the following files:

- func.h
- func.cpp

3.3 Xorshift32 Class Reference

Xorshift32 random number generator.

```
#include <func.h>
```

Public Member Functions

- Xorshift32 (uint32_t seed)

  Constructor that initializes the generator with a seed.
- uint32_t next ()

  Generates the next random number using xorshift algorithm.

3.3.1 Constructor & Destructor Documentation

3.3.1.1 Xorshift32()

```
Xorshift32::Xorshift32 (
            uint32_t seed )
```

Constructs a Xorshift32 generator with given seed.

Parameters

seed	Initial seed value

3.3.2 Member Function Documentation

3.3.2.1 next()

```
uint32_t Xorshift32::next ( )
```

Generates next random number using Xorshift algorithm.

Returns

Next 32-bit unsigned random number

32-bit unsigned random number

The documentation for this class was generated from the following files:

- func.h
- func.cpp

Chapter 4

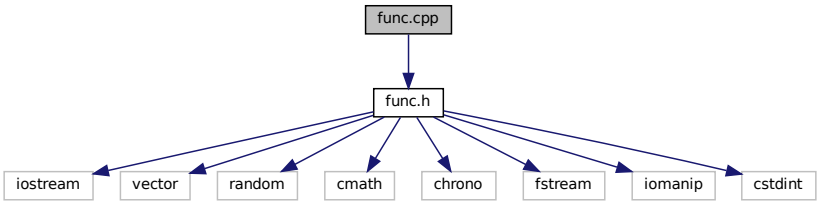
File Documentation

4.1 func.cpp File Reference

Implementation of random number generators and statistical tests for randomness analysis.

```
#include "func.h"
```

Include dependency graph for func.cpp:



Functions

- double incompleteGammaSeries (double a, double x)

  Calculates incomplete gamma function using series expansion.
- double incompleteGammaContinuedFrac (double a, double x)

  Calculates incomplete gamma function using continued fraction.
- double gammaQ (double a, double x)

  Computes regularized upper incomplete gamma function Q(a,x)
- double Phi (double x)

  Computes standard normal cumulative distribution function.
- double monobitTestP (const std::vector< int > &bits)

  Performs the monobit test on a binary sequence (NIST SP 800-22)
- double blockFrequencyTestP (const std::vector< int > &bits, int M=128)

  Performs the block frequency test on a binary sequence (NIST SP 800-22)
- double runsTestP (const std::vector< int > &bits)

  Performs the runs test on a binary sequence (NIST SP 800-22)
- double longestRunOnesTestP (const std::vector< int > &bits)

Performs the longest run of ones test on a binary sequence (NIST SP 800-22)

- double cumulativeSumsTestP (const std::vector< int > &bits)

  Performs the cumulative sums test on a binary sequence (NIST SP 800-22)
- void calculateStats (const std::vector< uint32_t > &values, double &mean, double &stddev, double &cov)

  Calculates basic statistics for a set of values.
- double chiSquareTestP (const std::vector< uint32_t > &values)

  Performs chi-square goodness-of-fit test on random numbers.
- std::vector< int > generateBits (const std::vector< uint32_t > &values)

  Converts 32-bit values to a sequence of bits.
- void performNISTTests (const std::vector< int > &bits, double &p1, double &p2, double &p3, double &p4, double &p5)

  Performs battery of NIST statistical tests on bit sequence.

4.1.1 Function Documentation

4.1.1.1 blockFrequencyTestP()

```
double blockFrequencyTestP (
            const std::vector< int > & bits,
            int M = 128 )
```

Performs the block frequency test on a binary sequence.

Parameters

bits	Vector of bits (0s and 1s) to test
M	Block size (default = 128)

Returns

P-value indicating the probability of randomness

Tests the proportion of ones within M-bit blocks to verify they are approximately 50%

4.1.1.2 calculateStats()

```
void calculateStats (
            const std::vector< uint32_t > & values,
            double & mean,
            double & stddev,
            double & cov )
```

Parameters

	values	Input vector of 32-bit unsigned integers
out	*mean*	Calculated mean value
out	*stddev*	Calculated standard deviation
out	*cov*	Calculated coefficient of variation (stddev/mean)

4.1.1.3 chiSquareTestP()

```
double chiSquareTestP (
            const std::vector< uint32_t > & values )
```

Performs chi-square goodness-of-fit test.

Parameters

values	Vector of 32-bit unsigned integers to test

Returns

P-value indicating the probability of uniform distribution

Tests if the numbers are uniformly distributed across the range

4.1.1.4 cumulativeSumsTestP()

```
double cumulativeSumsTestP (
            const std::vector< int > & bits )
```

Performs the cumulative sums test on a binary sequence.

Parameters

bits	Vector of bits (0s and 1s) to test

Returns

P-value indicating the probability of randomness

Tests the maximum deviation of the cumulative sum of the sequence from zero

4.1.1.5 gammaQ()

```
double gammaQ (
            double a,
            double x )
```

Calculates the regularized upper incomplete gamma function Q(a,x)

Parameters

a	Shape parameter
x	Upper limit of integration

Returns

Value of Q(a,x)

4.1.1.6 generateBits()

```
std::vector<int> generateBits (
            const std::vector< uint32_t > & values )
```

Parameters

values	Vector of 32-bit unsigned integers

Returns

Vector of bits (0s and 1s) extracted from the input values

Extracts all 32 bits from each input value (MSB first)

4.1.1.7 incompleteGammaContinuedFrac()

```
double incompleteGammaContinuedFrac (
            double a,
            double x )
```

Parameters

a	Shape parameter
x	Upper limit of integration

Returns

Computed value of incomplete gamma function

4.1.1.8 incompleteGammaSeries()

```
double incompleteGammaSeries (
            double a,
            double x )
```

Parameters

a	Shape parameter
x	Upper limit of integration

Returns

Computed value of incomplete gamma function

4.1.1.9 longestRunOnesTestP()

```
double longestRunOnesTestP (
            const std::vector< int > & bits )
```

Performs the longest run of ones test on a binary sequence.

Parameters

bits	Vector of bits (0s and 1s) to test

Returns

P-value indicating the probability of randomness

Tests the longest run of ones within M-bit blocks of the sequence

4.1.1.10 monobitTestP()

```
double monobitTestP (
            const std::vector< int > & bits )
```

Performs the monobit test on a binary sequence.

Parameters

bits	Vector of bits (0s and 1s) to test

Returns

P-value indicating the probability of randomness (values >= 0.01 suggest randomness)

Tests the proportion of ones and zeros in the sequence to verify they are approximately equal

4.1.1.11 performNISTTests()

```
void performNISTTests (
            const std::vector< int > & bits,
            double & p1,
            double & p2,
            double & p3,
            double & p4,
            double & p5 )
```

Performs a battery of NIST statistical tests on a bit sequence.

Parameters

	bits	Input bit sequence to test
out	*p1*	P-value from monobit test
out	*p2*	P-value from block frequency test
out	*p3*	P-value from runs test
out	*p4*	P-value from longest run of ones test
out	*p5*	P-value from cumulative sums test

4.1.1.12 Phi()

```
double Phi (
            double x )
```

Calculates the standard normal cumulative distribution function.

Parameters

x	Input value

Returns

Probability that a random variable is less than or equal to x

4.1.1.13 runsTestP()

```
double runsTestP (
            const std::vector< int > & bits )
```

Performs the runs test on a binary sequence.

Parameters

bits	Vector of bits (0s and 1s) to test

Returns

P-value indicating the probability of randomness

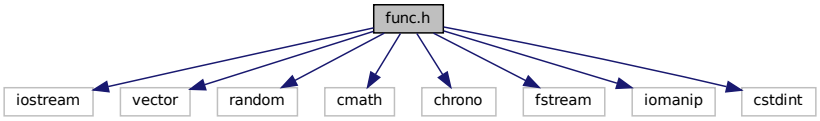
Tests the total number of runs (both 0-runs and 1-runs) in the sequence

4.2 func.h File Reference

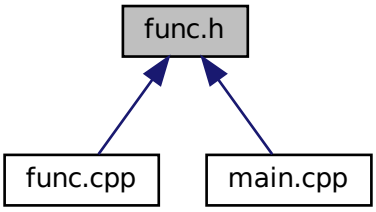
Declaration of random number generators (LCG, Xorshift32, MiddleSquare) and statistical test functions for randomness analysis.

```
#include <iostream>
#include <vector>
#include <random>
#include <cmath>
#include <chrono>
#include <fstream>
#include <iomanip>
#include <cstdint>
```

Include dependency graph for func.h:



This graph shows which files directly or indirectly include this file:



Classes

- class LCG

  Linear Congruential Generator (LCG) random number generator.
- class Xorshift32

  Xorshift32 random number generator.
- class MiddleSquare

  Middle Square random number generator.

Functions

- double incompleteGammaSeries (double a, double x)

  Calculates incomplete gamma function using series expansion.
- double incompleteGammaContinuedFrac (double a, double x)

  Calculates incomplete gamma function using continued fraction.
- double gammaQ (double a, double x)

  Calculates the regularized upper incomplete gamma function Q(a,x)
- double Phi (double x)

  Calculates the standard normal cumulative distribution function.
- double monobitTestP (const std::vector< int > &bits)

  Performs the monobit test on a binary sequence.
- double blockFrequencyTestP (const std::vector< int > &bits, int M)

  Performs the block frequency test on a binary sequence.
- double runsTestP (const std::vector< int > &bits)

  Performs the runs test on a binary sequence.
- double longestRunOnesTestP (const std::vector< int > &bits)

  Performs the longest run of ones test on a binary sequence.
- double cumulativeSumsTestP (const std::vector< int > &bits)

  Performs the cumulative sums test on a binary sequence.
- void calculateStats (const std::vector< uint32_t > &values, double &mean, double &stddev, double &cov)

  Calculates basic statistics for a set of values.
- double chiSquareTestP (const std::vector< uint32_t > &values)

  Performs chi-square goodness-of-fit test.
- std::vector< int > generateBits (const std::vector< uint32_t > &values)

  Converts 32-bit values to a sequence of bits.
- void performNISTTests (const std::vector< int > &bits, double &p1, double &p2, double &p3, double &p4, double &p5)

  Performs a battery of NIST statistical tests on a bit sequence.

4.2.1 Function Documentation

4.2.1.1 blockFrequencyTestP()

```
double blockFrequencyTestP (
            const std::vector< int > & bits,
            int M = 128 )
```

Parameters

bits	Vector of bits (0s and 1s)
M	Block size

Returns

P-value from the test

Performs the block frequency test on a binary sequence.

Parameters

bits	Vector of bits (0s and 1s) to test
M	Block size (default = 128)

Returns

P-value indicating the probability of randomness

Tests the proportion of ones within M-bit blocks to verify they are approximately 50%

4.2.1.2 calculateStats()

```
void calculateStats (
            const std::vector< uint32_t > & values,
            double & mean,
            double & stddev,
            double & cov )
```

Parameters

	values	Input values
out	*mean*	Calculated mean
out	*stddev*	Calculated standard deviation
out	*cov*	Calculated coefficient of variation
	values	Input vector of 32-bit unsigned integers
out	*mean*	Calculated mean value
out	*stddev*	Calculated standard deviation
out	*cov*	Calculated coefficient of variation (stddev/mean)

4.2.1.3 chiSquareTestP()

```
double chiSquareTestP (
            const std::vector< uint32_t > & values )
```

Parameters

values	Input values to test

Returns

P-value from the test

Performs chi-square goodness-of-fit test.

Parameters

values	Vector of 32-bit unsigned integers to test

Returns

P-value indicating the probability of uniform distribution

Tests if the numbers are uniformly distributed across the range

4.2.1.4 cumulativeSumsTestP()

```
double cumulativeSumsTestP (
            const std::vector< int > & bits )
```

Parameters

bits	Vector of bits (0s and 1s)

Returns

P-value from the test

Performs the cumulative sums test on a binary sequence.

Parameters

bits	Vector of bits (0s and 1s) to test

Returns

P-value indicating the probability of randomness

Tests the maximum deviation of the cumulative sum of the sequence from zero

4.2.1.5 gammaQ()

```
double gammaQ (
            double a,
            double x )
```

Parameters

a	Shape parameter
x	Upper limit of integration

Returns

Value of Q(a,x)

Calculates the regularized upper incomplete gamma function Q(a,x)

Parameters

a	Shape parameter
x	Upper limit of integration

Returns

Value of Q(a,x)

4.2.1.6 generateBits()

```
std::vector<int> generateBits (
            const std::vector< uint32_t > & values )
```

Parameters

values	Input values

Returns

Vector of bits (0s and 1s)

Parameters

values	Vector of 32-bit unsigned integers

Returns

Vector of bits (0s and 1s) extracted from the input values

Extracts all 32 bits from each input value (MSB first)

4.2.1.7 incompleteGammaContinuedFrac()

```
double incompleteGammaContinuedFrac (
            double a,
            double x )
```

Parameters

a	Shape parameter
x	Upper limit of integration

Returns

Value of the incomplete gamma function

Parameters

a	Shape parameter
x	Upper limit of integration

Returns

Computed value of incomplete gamma function

4.2.1.8 incompleteGammaSeries()

```
double incompleteGammaSeries (
            double a,
            double x )
```

Parameters

a	Shape parameter
x	Upper limit of integration

Returns

Value of the incomplete gamma function

Parameters

a	Shape parameter
x	Upper limit of integration

Returns

Computed value of incomplete gamma function

4.2.1.9 longestRunOnesTestP()

```
double longestRunOnesTestP (
            const std::vector< int > & bits )
```

Parameters

bits	Vector of bits (0s and 1s)

Returns

P-value from the test

Performs the longest run of ones test on a binary sequence.

Parameters

bits	Vector of bits (0s and 1s) to test

Returns

P-value indicating the probability of randomness

Tests the longest run of ones within M-bit blocks of the sequence

4.2.1.10 monobitTestP()

```
double monobitTestP (
            const std::vector< int > & bits )
```

Parameters

bits	Vector of bits (0s and 1s)

Returns

P-value from the test

Performs the monobit test on a binary sequence.

Parameters

bits	Vector of bits (0s and 1s) to test

Returns

P-value indicating the probability of randomness (values >= 0.01 suggest randomness)

Tests the proportion of ones and zeros in the sequence to verify they are approximately equal

4.2.1.11 performNISTTests()

```
void performNISTTests (
            const std::vector< int > & bits,
            double & p1,
            double & p2,
            double & p3,
            double & p4,
            double & p5 )
```

Parameters

	bits	Input bit sequence to test
out	*p1*	P-value from monobit test
out	*p2*	P-value from block frequency test
out	*p3*	P-value from runs test
out	*p4*	P-value from longest run of ones test
out	*p5*	P-value from cumulative sums test

Performs a battery of NIST statistical tests on a bit sequence.

Parameters

	bits	Input bit sequence to test
out	*p1*	P-value from monobit test
out	*p2*	P-value from block frequency test
out	*p3*	P-value from runs test
out	*p4*	P-value from longest run of ones test
out	*p5*	P-value from cumulative sums test

4.2.1.12 Phi()

```
double Phi (
            double x )
```

Parameters

x	Input value

Returns

Probability that a random variable is less than or equal to x

Calculates the standard normal cumulative distribution function.

Parameters

x	Input value

Returns

Probability that a random variable is less than or equal to x

4.2.1.13 runsTestP()

```
double runsTestP (
            const std::vector< int > & bits )
```

Parameters

bits	Vector of bits (0s and 1s)

Returns

P-value from the test

Performs the runs test on a binary sequence.

Parameters

bits	Vector of bits (0s and 1s) to test

Returns

P-value indicating the probability of randomness

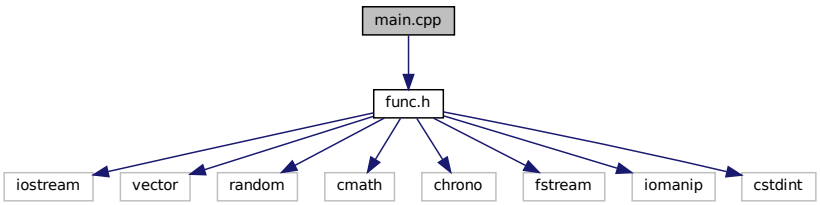
Tests the total number of runs (both 0-runs and 1-runs) in the sequence

4.3 main.cpp File Reference

Random Number Generator Testing Application.

```
#include "func.h"
```

Include dependency graph for main.cpp:



Functions

- int main ()

  Main function for random number generator testing.

4.3.1 Detailed Description

This program evaluates and compares four pseudo-random number generators:

1. Linear Congruential Generator (LCG)
2. Xorshift32
3. Middle Square Method
4. Mersenne Twister (std::mt19937)

The evaluation includes statistical tests and performance benchmarks.

4.3.2 Function Documentation

4.3.2.1 main()

```
int main ( )
```

Returns

Exit code (0 for success)

The program performs the following operations:

1. Generates test sequences for each PRNG algorithm
2. Saves sequences to files for further analysis
3. Conducts statistical analysis:
   - Basic statistics (mean, standard deviation, coefficient of variation)
   - Uniformity test (chi-square)
   - Randomness tests (NIST test suite)
4. Measures and compares generation performance
5. Outputs results in both human-readable and CSV formats

4.4 plt.py File Reference

Performance comparison visualization for random number generators.

Variables

- **plt.df** = pd.read_csv('times.csv')
- **plt.label**
- **plt.dpi**
- **plt.bbox_inches**

Index